Contract-Driven Harness Engineering for Reliable Low-Cost Agent Tasks

Contract-Driven Harness Study

June 10, 2026

Abstract

Many agent failures in productivity tasks are treated as model failures. In bounded tasks, some of those failures are more concrete: the model was not given an inspectable obligation to preserve evidence, state, stage gates, or output fields. We study contract-driven harness engineering: a reliability layer that represents those obligations as task specifications, bounded memory slices, evidence bundles, output contracts, validation gates, and trace requirements.

The study is deliberately narrow. It does not test whether harnessing makes low-cost models generally equivalent to strong models. It asks whether explicit obligations can turn failures observed in low-cost-model runs into inspectable, repairable, and regression-testable engineering objects.

Across structured extraction, project initialization, research workflow, mechanism-atom, and admitted-macro experiments, harnessing improves absolute contract adherence. It can also compress model gaps when baseline gaps are nonzero and the task is highly constrained. Gap compression is not universal. The more stable result is weak-model enablement on bounded, contract-critical operations.

The resulting method is mechanism-first: decompose broad workflows into testable mechanisms, admit only mechanisms with golden/known-bad/local-gate support, and expand macro scope only when carried obligations are explicit. In Stage 7e and Stage 7-next, repaired contract obligations let the low-cost model pass targeted smoke runs on fixed evidence-bound macros. The evidence supports bounded weak-model enablement. It does not support production readiness or open-ended workflow reliability.

1 Introduction

Agent systems are now used for ordinary productivity work: initializing projects, extracting structured information, synthesizing evidence, preparing plans, updating documents, and coordinating multi-step workflows. In these settings, reliability is often attributed directly to model capability. If an agent loses a constraint, cites no evidence, skips a stage, or reuses stale context, the common explanation is that the underlying model is not strong enough.

That explanation is sometimes correct, but it misses a system-design problem. Many productivity tasks contain obligations that can be stated before generation starts: which evidence may be used, which state is known or unknown, which actions are blocked, which fields must be present, which claims require citations, and which stage gate prevents a final recommendation. If those obligations stay implicit, the model must infer and retain them while generating. If they are represented as contracts, part of the reliability burden moves from the model into the surrounding system.

We call this approach contract-driven harness engineering. It uses task specifications, bounded memory, evidence bundles, output contracts, workflow gates, validators, and trace requirements

to constrain agent behavior. Low-cost models are not generally equivalent to strong models. The question studied here is smaller: which parts of agent reliability become less dependent on unconstrained generation once task obligations are made inspectable?

The study separates three problems that are often collapsed in agent evaluation. Model capability is the model's ability to reason, follow instructions, and recover from ambiguity. Harness specification is the system's ability to state task obligations, admissible evidence, known and unknown state, output structure, and blocked actions. Workflow composition is the ability to preserve those obligations across steps, tools, and state transitions. The current evidence covers harness specification and bounded composition. It does not cover open-ended workflow autonomy.

The project began with a gap-compression hypothesis: a stable harness may reduce the measured gap between strong and low-cost models. The results support that hypothesis only under specific conditions. In highly structured extraction tasks, stronger harnessing compressed measured nonzero baseline gaps. In broader project-initialization and research-workflow slices, harnessing improved absolute contract adherence but produced mixed or undefined gap movement. Those broader slices exposed a measurement problem. A workflow-level task can hide several mechanisms at once: schema following, state retention, evidence grounding, stage discipline, and trace completeness.

We therefore moved from workflow-first evaluation to mechanism-first evaluation. A mechanism atom is a fixed-input, deterministic, contract-bound operation with one primary mechanism, one dominant failure mode, a golden output, a known-bad output, and a composition interface. This yields smaller evaluation questions: Does an explicit evidence bundle improve claim grounding? Does a memory slice prevent state hallucination? Does a stage gate prevent premature recommendation? Does a trace requirement make rejected options auditable? Passing atoms do not prove that a whole workflow is solved, but they make later composition easier to interpret.

The strongest evidence in this study comes from the repair loop created after broad workflow tests exposed unstable behavior. In Stage 7e, we composed a narrow evidence-bound decision macro from state inventory, evidence grounding, evidence typing, traceable decision, and stage-gated synthesis mechanisms. The first version showed that a low-cost model could pass under one harness arm but still lose decision-trace and stage-gate obligations under another. We isolated the miss and revised the contract. Stage 7e v2 repaired trace and gate retention, but exposed unknown-state omissions. Stage 7e v3 repaired unknown-state retention, but exposed known-state provenance compression. Stage 7e v4 made known-state provenance explicit and passed 4/4 targeted smoke runs after retry.

We then checked whether this was only a one-fixture patch. Stage 7-next reused the Stage 7e v4 obligations in a neighboring evidence-bound method-plan update macro. The output had to select the next admitted macro, list admission criteria, preserve local and real-model gates, and declare non-claims. The low-cost model passed 4/4 targeted smoke runs under G8/G9, with all runs scoring 1.000 on task success and the strict primary macro metric. This is narrow evidence, but it shows that a repaired obligation set can transfer to a closely related bounded macro.

The resulting claim is narrower than the original gap-compression thesis. Contract-driven harnesses do not make weak models generally equivalent to strong models, and they do not guarantee gap compression. They can raise the usable floor of low-cost models on bounded, contract-critical operations. They also make some failures easier to repair: a missing obligation can be named, added to the contract, captured as a known-bad case, checked locally, rerun against a model, and carried into the evidence ledger and claim boundary.

1.1 Contributions

The paper makes five contributions:

1. We define contract-driven harness engineering as an explicit reliability layer for agent tasks.
2. We propose mechanism atoms as the unit of harness evaluation.
3. We report a multi-stage empirical evaluation across task slices, mechanism atoms, and admitted macros.
4. We introduce a repair-loop protocol for harness development.
5. We provide bounded evidence that low-cost models can reach pass-level contract adherence on fixed, evidence-bound, contract-critical macros.

2 Related Work

The relevant prior work spans agent orchestration, declarative LM programming, structured output constraints, retrieval and tool augmentation, memory systems, safety verification, and skill ecosystems. These lines of work share a practical move. They take obligations that would otherwise sit inside free-form model generation and place them in runtime, specification, tool, memory, validation, or evaluation layers.

The distinction is summarized in Table 1.

Work family	Main focus	What it externalizes	What this paper adds
Workflow orchestration	execution graph and state	steps, tools, persistence, human checkpoints	obligation-level evaluation and repair
Structured outputs	syntactic output control	schema and format constraints	semantic contract obligations such as evidence, unknown state, and blocked claims
Guardrails and validators	runtime checks and retries	validation policies and failure handling	known-bad-driven repair loops tied to mechanism atoms
Declarative LM programs	modules, signatures, metrics	program structure and optimization targets	mechanism-first empirical repair for low-cost-model enablement
Agent specifications	portability and interface contracts	workflow, state, and step definitions	evidence-bound admission criteria before macro composition
Retrieval, tools, and memory	external knowledge and actions	documents, APIs, tool calls, long-term state	bounded memory/evidence contracts before live side effects
Safety and verification	policy compliance and assurance	constraints, static checks, runtime firewalls	empirical contract adherence with explicit non-claims

2.1 Agent Workflows And Orchestration

Recent agent engineering guidance distinguishes between fully autonomous agents and workflows whose control paths are explicitly defined. Anthropic's discussion of effective agents is useful here: workflows fit tasks that can be decomposed into predictable steps, while agents fit cases that require open-ended model autonomy. LangGraph, AutoGen, Semantic Kernel, and related orchestration frameworks make a similar engineering move. They expose execution state, graph structure, persistence, human-in-the-loop intervention, tool calls, and observability as system concerns rather than leaving them solely inside a prompt. [2, 8, 12, 13]

These systems make execution durable, inspectable, and easier to integrate with tools and humans. That does not settle the evaluation problem in this study. A workflow graph can route steps correctly while still losing evidence provenance, collapsing unknown state, skipping stage gates, or producing an ungrounded final recommendation.

Contract-driven harness engineering overlaps with workflow orchestration but treats the graph as only one layer. The unit of interest is the obligation that must survive the graph: state inventory, evidence binding, evidence type separation, trace completeness, stage-gate retention, excluded-context preservation, and repairability.

2.2 Declarative LM Programs And Agent Specifications

Declarative LM programming systems, especially DSPy, argue that language model behavior should be represented as programs with signatures, modules, metrics, and optimizers rather than as hand-written prompts. Agent specification work such as AgentSPEX and AgentSpec pushes in a related direction for agent systems: workflows, state, steps, and interfaces should be declared in portable and inspectable forms. [7, 19, 1]

This line of work is close to our method. Both assume that some reliability gains come from making task structure explicit. The difference is the evaluation target. Declarative systems often emphasize program optimization, portability, or agent specification. Here, the unit is a contract-critical obligation. We define it, create golden and known-bad outputs, run deterministic local gates, execute small real-model slices, and update the claim boundary before composing broader workflows.

2.3 Structured Outputs, Guardrails, And Validators

Structured output systems and guardrail frameworks externalize output form. OpenAI structured output mechanisms, Outlines-style constrained generation, and Guardrails validators reduce the burden of asking a model to follow a format. They make schema adherence and selected validation checks part of the system layer. [14, 5, 6]

These mechanisms are necessary but insufficient for the problem studied here. Schema validity can guarantee that an output has the expected shape. By itself, it cannot guarantee that a claim is supported, that unknown state remains unknown, that excluded context is not reused, or that a recommendation is blocked when a stage gate is incomplete. The harness studied here treats validators as one component in a larger contract stack. The output contract includes fields, but also semantic obligations: required evidence IDs, evidence type separation, rejected-option traces, explicit blocked outputs, and non-claims.

2.4 Retrieval, Tools, Memory, And Externalized Capability

RAG, ReAct, Toolformer, Gorilla, and tool/API-focused agent work show that models can become more capable when knowledge and action are externalized. Retrieval can provide updated evidence and provenance. Tool-use frameworks can turn external actions into typed calls. API benchmarks show that tool descriptions and retrieval can materially improve call generation compared with unaided model behavior. [11, 21, 17, 16]

Memory-oriented systems such as MemGPT and Letta show that agents can use hierarchical, archival, or stateful memory to extend beyond a single context window. They also expose a reliability problem for this study: memory is not automatically beneficial. A system must decide what to store, when to summarize, how to retrieve, how to scope memory, and how to prevent stale or irrelevant context from contaminating a new task. [15, 10]

This study does not primarily test live retrieval, live tool execution, or long-term memory. Most admitted mechanism atoms and macros are fixed-input, no-tool tasks. That choice isolates whether the model can honor explicit contracts before live tools, changing corpora, or runtime side effects are added.

2.5 Evaluation, Safety, Verification, And Skill Ecosystems

Agent evaluation and safety work highlights the fragility of agent claims. OAgents-style critiques emphasize protocol variance and reproducibility challenges. Semantic Integrity Constraints, Agentproof, LlamaFirewall, and related verification or guardrail systems argue that agent behavior must be constrained, audited, or checked against explicit policies and semantic rules. [22, 9, 20, 3]

Capability ecosystems such as MCP servers, agent skills, registries, pack systems, and PEtFiSh-style skill markets represent another route toward harness engineering. They externalize reusable procedures, tool access, installation state, platform routing, quality gates, and capability discovery. In PEtFiSh specifically, packs, skills, MCP servers, installers, trigger evaluators, quality gates, and context plugins make the system a useful experimental setting for studying harness design as an engineering object. Local PEtFiSh-specific evidence is preserved in Appendix C and the reproducibility package.

PEtFiSh is the implementation context, not the transferable claim. The claim concerns a contract stack and repair-loop protocol: task specs, bounded memory, evidence bundles, output contracts, workflow gates, trace logs, validators, known-bad cases, and claim-boundary updates.

This paper focuses on a narrower gap: mechanism-level evaluation. Which explicit obligations let a low-cost model complete bounded contract-critical operations, and how should a harness be repaired when those obligations fail?

The closest prior lines are declarative LM programs, guardrail or validator systems, and agent specification languages. They externalize structure, interfaces, or runtime checks. Our contribution is operationally different: we treat each missing reliability obligation as an empirical repair target, require golden and known-bad fixtures before admission, and restrict macro claims to obligations that survive composition. The unit of evaluation is the obligation that must remain auditable across the harness, not the workflow graph or the schema alone.

3 Methods

3.1 Study Design

This study evaluates whether explicit agent harness contracts can make productivity tasks less dependent on unconstrained model behavior. Agent workflow is not treated as a primitive benchmark

unit. We evaluate harness mechanisms in three levels:

1. task slices, which compare broad task classes across harness strengths;
2. mechanism atoms, which isolate a single primary harness mechanism and a dominant failure mode;
3. admitted macros, which compose only mechanisms that have passed local gates and targeted model checks.

This design is intentionally conservative. Broad workflow results can motivate failure analysis, but they do not by themselves justify claims about a general harness methodology. A workflow can enter the main claim only when its component mechanisms, local evaluators, known-bad cases, and cross-step obligations are explicit.

The experimental path therefore moves from broad failures toward admitted composition rather than from one benchmark score to a larger benchmark score:

```
Task slices
  |- structured extraction
  |- project initialization
  `- research workflow
        -> failure analysis
Mechanism atoms
  |- Stage 6
  |- Stage 7r
  `- Stage 7r.1
        -> admission gate
Admitted macros
  |- Stage 7p / 7p v2
  |- Stage 7e v1-v4
  `- Stage 7-next
```

This map is the method in miniature. Broad tasks expose unstable behavior, mechanism atoms isolate the obligation, local gates reject known-bad outputs, and only then can a macro carry the obligation forward.

3.2 Harness Model

We define a contract-driven harness as a set of explicit control objects around a language model:

Object	Role
`TaskSpec`	Objective, constraints, success conditions, and non-goals.
`MemorySlice`	Bounded context that may be used, plus excluded or unknown state.
`EvidenceBundle`	Admissible evidence items, evidence types, and source links.
`OutputContract`	Required output shape, nested fields, citation policy, and validator rules.

Object	Role
`WorkflowGraph` or stage gate	Required order of intermediate steps and blocked outputs.
`TraceLog`	Decision trace requirements for auditable reasoning and rejection paths.
`ValidatorGate`	Deterministic local checks that distinguish passing outputs from known-bad outputs.

The study uses a conservative working assumption: when a task can be bounded, reliability should move from implicit model judgment into explicit, inspectable contracts. The harness is evaluated as a reliability-engineering layer, not as a claim that low-cost models become generally equivalent to strong models.

The object of study is not an improved prompt. It is the lifecycle by which a missing obligation becomes a contract field, a known-bad fixture, a deterministic gate, and an admitted macro requirement.

3.3 Harness Arms And Models

Experiments use harness arms to vary the strength of external control. G0 is raw or minimally constrained task input. G2/G3 represent intermediate mechanism arms when relevant. G8 is contract-rich execution with validator or evaluator obligations. G9 is the full harness packet with task spec, output contract, evidence, memory policy, workflow, trace, and regression expectations.

The real-model slices use SiliconFlow's OpenAI-compatible API. The current low-cost model tier is `Qwen/Qwen3-8B`; strong-model slices in earlier stages used `deepseek-ai/DeepSeek-V3.2`. Provider-backed runs use temperature `0`, fixed prompt artifacts exported before execution, per-run artifact directories, adapter request files, output files, validation reports, metrics files, and event logs for provider start/end, elapsed time, errors, and retry analysis. [18]

3.4 Task Slices, Mechanism Atoms, And Macros

The first empirical layer uses broad task slices to identify where harnessing helps and where broad task definitions become too noisy. Structured extraction tests a high-constraint task with deterministic output structure. Project initialization tests a multi-constraint workspace planning task. Research workflow tests evidence-backed synthesis.

A mechanism atom is the smallest testable unit of harness behavior with one primary mechanism, fixed input, explicit output contract, deterministic evaluator, known-bad rejection, pass threshold, and composition interface. Atoms are accepted only when fixture structure validates, the golden output passes, at least one known-bad output fails for the intended reason, the baseline leaves room for improvement, the low-cost model improves under the relevant harness arm, and the atom declares how it can compose with downstream mechanisms.

Macro tasks are composed only after component mechanisms pass local gates and targeted model checks. A macro must preserve cross-step obligations explicitly. The current admitted macro family is evidence-bound and fixed-input: Stage 7p v2, Stage 7e v1-v4, and Stage 7-next. Project initialization and full research workflow remain blocked because the current evidence covers bounded macros, not open-ended tool-using workflows.

3.5 Admission Criteria

An atom or macro is admitted to the next experimental layer only if all of the following hold:

1. the fixture schema validates;
2. the golden output passes;
3. at least one known-bad output fails for the intended reason;
4. the baseline leaves improvement room or the evaluation question explicitly targets absolute contract adherence;
5. the low-cost model improves under G8/G9 or reaches the declared pass threshold;
6. the composition interface is declared;
7. cross-step carried obligations are explicit when a macro composes multiple atoms;
8. unsupported claims and non-claims are updated before expansion.

These criteria are deliberately stricter than ordinary prompt evaluation. They are designed to prevent a broad workflow result from entering the claim set before the underlying mechanism and failure mode are visible.

3.6 Repair-Loop Protocol

The main methodological contribution is the repair-loop protocol:

1. observe a real model failure;
2. isolate the missing mechanism or obligation;
3. make the obligation explicit in the input and output contract;
4. add or update a known-bad fixture that captures the failure;
5. run local golden/bad regression;
6. execute a targeted real-model slice;
7. update the evidence ledger, claim boundary, and backlog before expanding scope.

Stage 7e illustrates this protocol. Stage 7e v1 found a low-cost-model G9 failure in stage-gate and decision-trace retention. Stage 7e v2 repaired trace/gate retention but exposed unknown-state omissions. Stage 7e v3 repaired unknown-state retention but exposed known-state provenance compression. Stage 7e v4 made known-state provenance explicit and passed 4/4 targeted smoke runs after retry. Stage 7-next then reused those obligations in a method-plan update macro and passed 4/4 targeted smoke runs without provider errors.

3.7 Metrics And Claim Rules

Each evaluated run emits metrics including `task_success`, `schema_validity`, `citation_grounding`, `state_accuracy`, `evidence_type_accuracy`, `stage_completion`, `trace_completeness`, `context_relevance`, and `atom_primary_metric`.

Table 2 gives a compact reading guide for the core metrics. Detailed thresholds, fixture-specific checks, and evaluator outputs are provided in the reproducibility package.

Metric	What it checks	Evaluator type
`schema_validity`	Required fields and types are present	deterministic schema/field check
`citation_grounding`	Claims carry admissible evidence IDs	deterministic evidence-ID check
`state_accuracy`	Known and unknown state are preserved	fixture-specific state check
`evidence_type_accuracy`	Evidence type labels remain correct	evidence-type check
`stage_completion`	Required stage gates are preserved	stage-gate check
`trace_completeness`	Required decision and rejection traces are present	structured trace check
`context_relevance`	Required or excluded context obligations are respected	context-obligation check
`atom_primary_metric`	The atom-specific dominant obligation is satisfied	atom evaluator
`task_success`	The declared task contract passes	composite evaluator

These metrics evaluate contract adherence rather than open-ended output quality or human preference.

Gap compression is computed only where a nonzero G0 baseline gap exists. If G0 baselines collapse for both models, or if the low-cost model improves more than the strong model and reopens the absolute gap in the opposite direction, the result is reported as mixed, undefined, or negative rather than forced into a compression claim.

Weak-model enablement is reported when the low-cost model reaches a pass threshold under a harness condition after failing or underperforming under weaker conditions.

4 Results

4.1 Overview

The results support a bounded version of the original hypothesis. Contract-rich harnessing improves absolute contract adherence across several productivity-task settings. It can also compress cross-model gaps in highly constrained tasks when baseline gaps are nonzero. Gap compression is not universal. The more stable result is weak-model enablement on bounded, contract-critical operations: when task obligations are explicit and evaluated deterministically, the low-cost model can reach pass-level behavior on tasks where weaker harnessing or broader prompts were unstable.

Table 3 summarizes the main claim boundary.

Claim	Evidence	Boundary
Absolute contract adherence lift	broad slices, mechanism atoms, admitted macros	tested conditions only
Gap compression	strongest in structured extraction	conditional on nonzero baseline gaps and constrained tasks
Weak-model enablement	Stage 6, Stage 7r.1, Stage 7e v4, Stage 7-next	bounded contract-critical operations
Full workflow reliability	not supported	remains a non-claim

Claim	Evidence	Boundary
Production readiness	not supported	remains a non-claim

Table 4 summarizes the main empirical layers and the claim each layer permits.

Layer	Stage	Runs	Failure -> Repair	Outcome	Allowed claim
Task slice	structured extraction	24	schema/tool gaps -> G9 packet	key gaps to 0	conditional gap compression
Task slice	project init	12	mixed metric movement -> G9	mixed	no universal compression
Task slice	research workflow	12	zero or mixed baseline gaps -> G9	mixed/undefined	absolute adherence only
Atom	Stage 6	48	mechanism failures -> atom contracts	low-cost lift	weak-model enablement
Macro	Stage 7p/v2	12	stale-context loss -> carried obligations	repaired in v2	composition retention
Macro	Stage 7e	6 + repairs	trace and state gaps -> explicit obligations	targeted smoke passed	fixed macro only
Macro	Stage 7-next	4	neighboring transfer -> reused obligations	targeted smoke passed	narrow transfer

4.2 Task Slices: Strong Absolute Lift, Conditional Gap Compression

The structured-extraction v2 slice provides the clearest positive gap-compression result. Under G0, nonzero baseline gaps appeared on task success, schema validity, tool-call correctness, human acceptance, cost efficiency, and safety consistency. Under G9, all measured nonzero gaps compressed to 0.000, producing compression ratios of 1.000 on those metrics. Citation grounding had a baseline gap of 0.000 and is therefore reported as n/a rather than as compression.

The project-initialization slice shows why gap compression cannot be the universal claim. G9 compressed the task-success gap from 0.111 to 0.000 and the safety-consistency gap from 0.200 to 0.000. Schema validity moved in the opposite direction, from a baseline gap of 0.250 to an arm gap of 0.583, yielding a negative compression ratio of -1.333. Human acceptance and cost efficiency also showed negative compression ratios.

The research-workflow slice further weakens a universal gap-compression story. Several G0 baseline gaps were already 0.000, making compression undefined. G9 compressed schema-validity gap from 0.067 to 0.000, but task-success gap became 0.083 from a 0.000 baseline and human-acceptance/cost-efficiency gap movement was slightly negative.

These results are best interpreted as absolute lift with conditional gap compression, not universal gap closure.

4.3 Mechanism Atoms: Broad Workflows Need Smaller Units

The Stage 6 mechanism-atom pilot tested 48 real model runs and completed 48/48 after documented timeout recovery. Its strongest result was weak-model enablement. For the low-cost model relative to G0, G9 produced task_success lift of +0.576 and schema_validity and atom_primary_metric lift of +0.833.

The harnessed low-cost model also outperformed the unconstrained strong model on contract-critical metrics. Relative to strong_model + G0, low-cost model + G9 showed task_success advantage of +0.743 and schema_validity and atom_primary_metric advantages of +1.000.

Gap compression was mostly positive on general contract metrics, including G9 task_success compression of 1.000 and G9 schema_validity compression of 1.000. But atom_primary_metric remained mixed, with G9 compression of 0.000 and G2/G8 negative values. This supports the conclusion that harnesses can substantially raise low-cost-model reliability on bounded operations even when atom-specific gap compression is not uniform.

4.4 Composition: Atom Success Is Not Enough

Stage 7p tested whether passing atoms could compose into a narrow partial macro:

```
A10 bounded context recall -> A9 no-overwrite action planning -> A6 validator repair
```

The execution completed cleanly with 6/6 real SiliconFlow runs. Strong_model G8 and G9 passed the full partial-composition chain. The low-cost model improved strongly under G8/G9, reaching task_success=0.800 under G8 and 0.900 under G9, with schema_validity=1.000 and safety=1.000. Yet it failed the full chain because context_relevance stayed 0.000: the composed output did not explicitly carry forward the stale-context exclusion.

Stage 7p v2 then added an explicit composition-retention contract. The same partial chain passed for both model tiers under G8/G9. This result supports a composition-layer mechanism claim: cross-step obligation retention is necessary when a macro must preserve negative context constraints across multiple atom outputs.

4.5 Revised Atoms And Targeted Repairs

Stage 7r redesigned six boundary-prone atoms: A2R, A3R, A4R, A5R, A7R, and A8R. Local gates passed: 6/6 fixture structures, 12/12 local golden/bad expectations, 36/36 packet compilation, and preflight with 0 errors and 0 warnings.

The real-model smoke completed 35/36 outputs. The single missing output was A8R low-cost G8, which repeatedly timed out under SiliconFlow and was treated as an execution deviation rather than a model-quality score. On completed runs, strong-model G8/G9 passed 12/12, while the low-cost model still failed strict A2R citation grounding and A7R trace completeness.

Stage 7r.1 targeted exactly those failures by tightening the contracts. A2R1 required every grounded claim to be an object with non-empty `evidence_ids`. A7R1 required rejected-option objects with evidence IDs and trace steps for C2 support, C1 rejection, and C3 rejection. The targeted 8-run low-cost-model smoke passed 8/8.

This supports the mechanism-first repair hypothesis: low-cost-model failures on claim-level evidence binding and rejection-trace completeness can be repaired by narrowing the output contract.

4.6 Stage 7e: Iterative Repair Of A Fixed Evidence-Decision Macro

Stage 7e composed a narrow evidence-bound decision macro from state inventory, evidence grounding, evidence-type separation, traceable decision, and stage-gated synthesis mechanisms. The first Stage 7e smoke completed 6/6 runs. Strong_model G8/G9 and low-cost-model G8 passed with task_success=1.000 and atom_primary_metric=1.000. G0 failed for both model tiers. Low-cost-model G9 partially failed with task_success=0.714 because it missed complete decision-trace and stage-gate retention.

Stage 7e v2 added explicit retention requirements for decision_trace, stage_gate, and carried_obligations. The targeted low-cost-model G8/G9 smoke completed 4/4 targeted smoke runs,

and all four targeted runs achieved trace_completeness=1.000 and stage_completion=1.000. However, only 1/4 runs fully passed because the remaining runs omitted Git branch, CI status, or network/API approval unknowns from state_inventory.

Stage 7e v3 targeted unknown-state retention. The four-run targeted smoke preserved all required Git/CI/network unknown-state fields and forbidden-inference fields. Full strict macro pass improved to 3/4. The remaining failure was narrower: one G8 run compressed known-state provenance into generic labels.

Stage 7e v4 made known-state provenance explicit. It required `state_inventory.known_state[]` entries with `state_id`, `fact`, and `evidence_ids`. After retrying one provider timeout and one truncated-output retry, all four targeted runs passed with task_success=1.000 and atom_primary_metric=1.000. State accuracy, citation grounding, evidence type accuracy, trace completeness, and stage completion all reached 1.000.

This sequence is the clearest evidence for the repair-loop protocol. The low-cost model did not become generally stronger. The harness made the missing obligations explicit, then verified that the model could satisfy them under a fixed macro contract.

4.7 Stage 7-next: Transfer To A Neighboring Macro

Stage 7-next tested whether the Stage 7e v4 obligation set was only a fixture-specific patch. The new macro was an evidence-bound method-plan update. It reused Stage 7e v4 obligations and added one new stressor: the output had to select the next admitted macro, specify admission criteria, preserve local and real-model gates, and declare non-claims.

The local gate passed with 2/2 expectations met: the golden output passed with task_success=1.000 and atom_primary_metric=1.000, while the known-bad premature broader-workflow expansion failed with task_success=0.000.

The real smoke used the low-cost model only:

```
Qwen/Qwen3-8B x G8/G9 x 2 repetitions = 4 runs
```

All four targeted runs executed without provider errors, timeout, or truncated-output retry. The same slice passed with task_success=1.000, atom_primary_metric=1.000, schema_validity=1.000, citation_grounding=1.000, state_accuracy=1.000, evidence_type_accuracy=1.000, trace_completeness=1.000, stage_completion=1.000, and context_relevance=1.000.

This supports a narrow transfer claim: the Stage 7e v4 obligations can be reused to make the low-cost model complete a closely related fixed method-plan macro with one new explicit stressor.

5 Discussion And Limitations

5.1 What The Results Mean

The main finding is that some reliability requirements can be moved out of the model and into explicit contracts. When task state, admissible evidence, output shape, stage gates, trace requirements, and carried obligations are inspectable objects, low-cost models can satisfy bounded tasks that were unstable under weaker or broader prompts.

This changes how we interpret several failures in the experiments. The low-cost model did not preserve decision-trace retention until the trace was made structurally required. It did not preserve unknown state until unknown state was enumerated. It compressed known-state provenance until

provenance-bearing state objects were required. It failed partial macro composition until cross-step carried obligations were made explicit. These are not just weaker answers. They are missing obligations that can be named and tested.

5.2 Gap Compression Is Conditional

The original research motivation was model capability gap compression. The results support that idea in structured settings, but not as a general law. Structured extraction produced the clearest compression result because the input, output, and correctness criteria are highly constrained. Project initialization and research workflow produced a more complicated pattern: harnessing improved absolute contract adherence, but gap movement was mixed, undefined, or negative depending on the metric.

The implication is that gap compression is an observed outcome, not the thesis. It depends on the task, metric, baseline gap, and harness arm. The more useful question is whether the low-cost model reaches a declared contract-adherence threshold under explicit harnessing.

5.3 Why Negative Results Matter

Several negative or partial results carry real information. Stage 7p v1 showed that passing atoms do not automatically compose. Stage 7r showed that redesigned atoms could improve low-cost-model performance without fully repairing all contract-critical behavior. Stage 7e v2 and v3 exposed the missing obligations that made Stage 7e v4 meaningful.

The negative results are therefore part of the method, not noise around it.

5.4 Bounded Macros Are Not Full Workflows

Stage 7e v4 and Stage 7-next are fixed-input, no-tool, deterministic tasks. They do not involve live source discovery, file mutation, external tool execution, or changing workspace state. That boundary matters for how the results should be read.

The result should be stated carefully:

> Low-cost models can complete bounded evidence-bound macros when reliability obligations are explicit.

It should not be stated as:

> Low-cost models can reliably run full project initialization or full research workflows.

Full workflows introduce problems that the current macros do not test: tool selection, permission handling, filesystem mutation, live source volatility, partial failure recovery, long-horizon memory, user clarification, and multi-step state updates. The current work provides a method for approaching those workflows, not evidence that they are already solved.

5.5 Deterministic Evaluation, Sample Size, And Runtime Effects

The evaluation pipeline relies on deterministic evaluators, golden outputs, and known-bad outputs. This makes pass/fail decisions auditable and regression-testable. It also keeps the benchmark away from subjective preference scoring. The tradeoff is narrower validity: the metrics capture contract adherence, not prose quality, human usefulness, creative insight, or open-ended judgment.

The evaluator itself can also overfit. A known-bad suite covers only the failure modes that were anticipated or observed. A harness can pass the current deterministic gate and still fail under semantically equivalent field variants, evidence perturbations, or new tool-state conditions. The next evaluation layer should include perturbation suites before broader workflow claims, including field-name perturbations, evidence-order perturbations, semantically equivalent unknown-state variants, distractor evidence, and tool-state changes.

Several later experiments are targeted smoke tests with small run counts. Stage 7e v4 used four targeted smoke runs after retry. Stage 7-next used four targeted smoke runs. These are acceptable for mechanism repair but not for population-level performance estimation.

Provider behavior also affected experiments. Some SiliconFlow runs timed out, and Stage 7e v4 required retrying one timed-out run and one truncated-output retry before completion. Stage 7-next did not require retry and completed 4/4 targeted smoke runs cleanly. Runtime deviations should be reported as validity threats rather than hidden.

The present evidence uses SiliconFlow-backed runs and should be replicated across providers and model families before making provider-independent claims. This paper therefore does not make a provider-independent reliability claim.

5.6 PEtFiSh Specificity And Harness Cost

The fixtures and workflows are grounded in the PEtFiSh project context. The exact skills, packs, evidence ledgers, and backlog structures may not generalize to all agent systems. For this reason, PEtFiSh should be read as the implementation context. The transferable constructs are task specs, bounded memory, evidence bundles, output contracts, workflow gates, trace logs, validators, known-bad cases, and repair loops.

The claim is about the contract stack and the repair-loop protocol, not the specific pack catalog, skill names, or project directory conventions.

Contract-driven harnessing also adds overhead. It requires fixture design, schemas, evidence bundles, evaluators, local gates, manifests, event logs, and postprocessing. For simple tasks, this overhead may exceed the benefit. There is also a risk of over-constraint: strong models may become more stable but less flexible under strict contracts.

5.7 Future Work

The next empirical step is to build the next admitted macro only after identifying the required mechanisms and cross-step obligations. Promising directions include citation-bound synthesis, state-mutating project macros, live research workflow slices, cross-provider replication, and perturbation suites for field names, evidence order, unknown-state variants, distractor evidence, and tool-state changes.

6 Conclusion

Contract-driven harness engineering makes some low-cost-model failures observable, repairable, and regression-testable by moving reliability obligations into explicit contracts. Model quality still matters. The point is narrower: for bounded productivity tasks, part of agent reliability can be engineered outside the model.

The supported claim is weak-model enablement on bounded contract-critical operations. Gap compression remains conditional. It is clearest in structured extraction and present on some con-

tract metrics in mechanism and partial macro tests. It is mixed or undefined in broader project-initialization and research-workflow slices.

The practical value is the repair loop. A harness failure can be turned into a named obligation, a contract revision, a known-bad fixture, a local regression gate, a targeted real-model smoke test, and an updated claim boundary.

7 Appendix A. Current Non-Claims

This paper should not claim:

- low-cost models are generally equivalent to strong models;
- harnessing universally compresses model gaps;
- full project initialization is solved;
- full research workflow is solved;
- the harness is production ready;
- fixed macro success implies open-ended tool-using workflow reliability.

8 Appendix B. Contribution-To-Evaluation Alignment

Contribution	Evaluation support	Boundary
Contract-driven harness model	Task slices and methods artifacts	Method definition, not a production-readiness claim.
Mechanism atoms	Atom definition, coverage framework, Stage 6-7 atom results	Atom pass does not prove workflow pass.
Conditional gap compression	Structured extraction, project initialization, research workflow slices	Compression only when baseline gaps are nonzero and gap movement is not reversed.
Repair-loop protocol	Stage 7e v1-v4	Fixed evidence-bound decision macro.
Bounded weak-model enablement	Stage 7e v4 and Stage 7-next	Fixed-input, no-tool, deterministic macros.

9 Appendix C. Evidence Traceability Matrix

Paper claim	Evidence IDs	Source IDs	Status
Contract-rich harnessing improves absolute contract adherence and can compress gaps under constrained conditions.	P2-E28, P2-E30, P2-E32, P2-E33	P2-SILICONFLOW-V2-FULL24, P2-SILICONFLOW-PROJECT-INIT-12, P2-SILICONFLOW-RESEARCH-WORKFLOW-12, P2-CLAIM-BOUNDARY-MEMO	Supported with conditional wording.
Structured extraction is the clearest positive gap-compression task slice.	P2-E27, P2-E28	P2-SILICONFLOW-V2-FULL24	Supported for tested SiliconFlow v2 slice.
Project initialization and research workflow do not support universal gap compression.	P2-E30, P2-E32, P2-E33	P2-SILICONFLOW-PROJECT-INIT-12, P2-SILICONFLOW-RESEARCH-WORKFLOW-12, P2-CLAIM-BOUNDARY-MEMO	Supported; use mixed/undefined wording.
Mechanism atoms make broad workflow failures interpretable.	P2-E35, P2-E36, P2-E56, P2-E60	P2-MECHANISM-ATOM-DEFINITION, P2-MECHANISM-ATOM-COVERAGE, P2-STAGE7R-REVISED-ATOMS, P2-STAGE7R1-A2R-A7R-SMOKE	Supported as methodology and targeted empirical repair evidence.
Atom success does not automatically imply macro composition success.	P2-E51, P2-E52	P2-STAGE7P-PARTIAL-COMPOSITION	Supported by Stage 7p v1 failure.
Explicit cross-step carried obligations can repair the Stage 7p composition failure.	P2-E53, P2-E54	P2-STAGE7P-V2-COMPOSITION-RETENTION	Supported for A10 -> A9 -> A6 partial macro.
Stage 7r.1 repaired low-cost-model A2R/A7R failures through tighter output contracts.	P2-E57, P2-E58, P2-E59, P2-E60	P2-STAGE7R1-A2R-A7R-PREP, P2-STAGE7R1-A2R-A7R-SMOKE	Supported for targeted atoms only.

Paper claim	Evidence IDs	Source IDs	Status
Stage 7e v1-v4 demonstrates a repair-loop protocol for a fixed evidence-decision macro.	P2-E62, P2-E64, P2-E66, P2-E68, P2-E69, P2-E70	P2-STAGE7E-EVIDENCE-DECISION, P2-STAGE7E-V2-RETENTION, P2-STAGE7E-V3-STATE-RETENTION, P2-STAGE7E-V4-KNOWN-STATE-PROVENANCE, P2-CLAIM-BOUNDARY-MEMO	Supported with fixed-input/no-tool boundary.
Stage 7-next transfers Stage 7e v4 obligations to a neighboring method-plan macro.	P2-E72, P2-E74, P2-E75	P2-STAGE7-NEXT-METHOD-PLAN-LOCAL, P2-STAGE7-NEXT-METHOD-PLAN-SMOKE	Supported as narrow transfer evidence.
Full project initialization, full research workflow, production readiness, and general model equivalence remain non-claims.	P2-E33, P2-E63, P2-E69, P2-E70, P2-E75	P2-CLAIM-BOUNDARY-MEMO, P2-STAGE7E-EVIDENCE-DECISION, P2-STAGE7E-V4-KNOWN-STATE-PROVENANCE, P2-STAGE7-NEXT-METHOD-PLAN-SMOKE	Supported as explicit boundary.
Related work: orchestration, declarative programs, structured outputs, retrieval/tools, memory, verification, and skill ecosystems are adjacent lines.	P2-E05, P2-E06, P2-E07, P2-E08, P2-E09, P2-E83, P2-E84, P2-E85, P2-E86, P2-E87, P2-E88, P2-E89, P2-E90, P2-E91, P2-E92, P2-E93, P2-E94, P2-E95, P2-E96, P2-E98	External source IDs listed in `source-index.md`	Supported as background; convert to publication-style citations before submission.

10 Reproducibility Package

This draft is supported by a public reproducibility package in the project repository and by local artifacts under `research/` [4]. The package includes the source index, evidence ledger, mechanism-atom definitions, macro fixtures, prompt manifests, provider event logs, model-output artifacts, deterministic evaluator outputs, metric summaries, stage reports, citation audit reports, and citation metadata.

Repository: `https://github.com/kylecui/contract-driven-harness-study`.

A reviewer can audit the claims in four steps:

1. Read Appendix C to map each paper claim to evidence IDs.
2. Open `research/03_evidence/evidence-ledger.jsonl` and locate those evidence IDs.
3. Inspect the referenced stage report, metric summary, fixture, validator output, provider event log, and model-output artifact.
4. Re-run the deterministic local gate for the corresponding fixture when a fixture/evaluator pair is provided.

For example, the Stage 7e v4 claim starts from P2-E68, P2-E69, and P2-E70. A reviewer should then inspect the Stage 7e v4 report, macro fixture, validator output, provider event log, and metrics file before treating the claim as supported.

When available, stage reports include the exact command or script path used to regenerate local evaluator outputs. The repository README and method scripts provide the current entry points for local gate reruns and artifact inspection.

The core traceability files are:

- `research/01_sources/source-index.md`
- `research/01_sources/contract-driven-harness-citation-metadata.md`
- `research/03_evidence/evidence-ledger.jsonl`
- `research/06_outputs/contract-driven-harness-compact-results-appendix.md`
- `research/07_reviews/contract-driven-harness-citation-audit.md`
- `research/07_reviews/contract-driven-harness-source-coverage.md`
- `research/07_reviews/contract-driven-harness-unsupported-claims.md`

External references are prepared in `research/06_outputs/contract-driven-harness-references.bib`. Local empirical claims should be checked against Appendix C and the evidence ledger rather than treated as ordinary literature citations.

11 Bibliography

The BibTeX bibliography for this working draft is maintained in `research/06_outputs/contract-driven-harnes bib`.

For arXiv preparation, compile this manuscript with that BibTeX file and keep Appendix C as the evidence traceability layer. For ACM or IEEE submission, move most local evidence IDs to supplementary material and cite the local artifact bundle as a reproducibility package.

References

[1] Soufiane Amini et al. Open agent specification (agent spec): A unified representation for ai agents, 2025. arXiv v4 as of 2025-11-07.

[2] Anthropic. Building effective agents. `https://www.anthropic.com/engineering/building-effective-agents`. Accessed 2026-06-09.

[3] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 2025. arXiv v1 as of 2025-05-06.

[4] Contract-Driven Harness Study. Local experiment artifacts and evidence ledger. Local reproducibility package under research/, 2026. Includes source index, evidence ledger, stage reports, metrics files, event logs, macro fixtures, and paper drafts.

[5] dottxt-ai. Outlines documentation. `https://dottxt-ai.github.io/outlines/latest/`. Accessed 2026-06-09.

[6] Guardrails AI. Validators documentation. `https://guardrailsai.com/docs/concepts/validators/`. Accessed 2026-06-09.

[7] Omar Khattab et al. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023.

[8] LangChain. Langgraph documentation. `https://docs.langchain.com/oss/python/langgraph`. Accessed 2026-06-09.

[9] Alexander W. Lee, Justin Chan, Michael Fu, Nicolas Kim, Akshay Mehta, Deepti Raghavan, and Ugur Cetintemel. Semantic integrity constraints: Declarative guardrails for ai-augmented data processing systems, 2025. arXiv v3 as of 2025-08-07; also PVLDB 18(11):4073–4080, 2025.

[10] Letta. Agents overview documentation. `https://docs.letta.com/guides/agents/overview/`. Accessed 2026-06-09.

[11] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks, 2020.

[12] Microsoft. Autogen documentation. `https://microsoft.github.io/autogen/`. Accessed 2026-06-09.

[13] Microsoft. Semantic kernel documentation. `https://learn.microsoft.com/en-us/semantic-kernel/`. Accessed 2026-06-09.

[14] OpenAI. Structured outputs guide. `https://platform.openai.com/docs/guides/structured-outputs`. Accessed 2026-06-09.

[15] Charles Packer et al. MemGPT: Towards LLMs as operating systems, 2023.

[16] Shishir G. Patil et al. Gorilla: Large language model connected with massive APIs, 2023.

[17] Timo Schick et al. Toolformer: Language models can teach themselves to use tools, 2023.

[18] SiliconFlow. Chat completions and model documentation. `https://docs.siliconflow.cn/cn/api-reference/chat-completions/chat-completions`. Accessed 2026-06-09; model list also consulted at https://docs.siliconflow.cn/cn/userguide/models.

[19] Pengcheng Wang et al. AgentSPEX: An agent specification and execution language, 2026.

[20] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. Agentproof: Static verification of agent workflow graphs, 2026. Submitted 2026-03-20.

[21] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models, 2022.

[22] He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. OAgents: An empirical study of building effective agents, 2025. arXiv v2 as of 2025-06-23.